

JSON: Rozkład Zadań Web3

```
{
  "tasks":",
  "component": "root",
  "type": "fullstack",
  "action_type": "create",
  "depends_on":,
  "outputs":,
  "estimated_complexity": "low",
  "acceptance_criteria":
},
{
  "task_id": "INFRA-BE-SKELETON",
  "title": "Utworzenie szkieletu aplikacji backendowej w NestJS wewnątrz monorepo",
  "description": "Wygenerowanie nowej aplikacji NestJS w katalogu apps/backend. Aplikacja ta będzie sercem logiki biznesowej TipJar, obsługując API, autentykację, płatności i integracje. [1]",
  "component": "apps/backend",
  "type": "backend",
  "action_type": "create",
  "depends_on":,
  "outputs":,
  "estimated_complexity": "low",
  "acceptance_criteria":
},
{
  "task_id": "INFRA-FE-SKELETON",
  "title": "Utworzenie szkieletu aplikacji frontendowej w Next.js wewnątrz monorepo",
  "description": "Wygenerowanie nowej aplikacji Next.js w katalogu apps/frontend. Będzie ona odpowiedzialna za renderowanie interfejsu użytkownika, w tym strony głównej, paneli twórców i publicznych profili. [1]",
  "component": "apps/frontend",
  "type": "frontend",
  "action_type": "create",
```

```

"depends_on":,
"outputs":,
"estimated_complexity": "low",
"acceptance_criteria":
},
{
"task_id": "INFRA-SHARED-TYPES",
"title": "Stworzenie współdzielonej biblioteki typów TypeScript",
"description": "Utworzenie biblioteki w libs/types, która będzie zawierać
definicje typów (interfejsy, DTOs, enums) dla modeli danych, takich jak User i
Tip. Zapewni to spójność typów między backendem a frontendem,
minimalizując ryzyko błędów integracyjnych. [1]",
"component": "libs/types",
"type": "fullstack",
"action_type": "create",
"depends_on":,
"outputs":,
"estimated_complexity": "low",
"acceptance_criteria":
},
{
"task_id": "INFRA-DOCKER-COMPOSE",
"title": "Konfiguracja pliku docker-compose.yml dla lokalnego środowiska
deweloperskiego",
"description": "Stworzenie pliku docker-compose.yml, który definiuje i uruchamia
wszystkie niezbędne usługi do lokalnego rozwoju: bazę danych PostgreSQL,
Redis dla kolejek zadań oraz opcjonalnie kontenery dla aplikacji backendowej i
frontendowej. Umożliwi to szybkie i spójne uruchomienie całego środowiska
jednym poleceniem. [1]",
"component": "docker-compose.yml",
"type": "fullstack",
"action_type": "configure",
"depends_on":,
"outputs":,
"estimated_complexity": "medium",
"acceptance_criteria":
},
{

```

```

"task_id": "INFRA-CI-PIPELINE-SETUP",
"title": "Konfiguracja podstawowego pipeline'u CI (GitHub Actions / GitLab CI)",
"description": "Stworzenie pliku konfiguracyjnego dla CI (np.
.github/workflows/ci.yml ), który będzie automatycznie uruchamiany przy każdym
pushu. Pipeline powinien instalować zależności, uruchamiać linting (ESLint)
oraz budować obie aplikacje (backend i frontend) w celu weryfikacji
poprawności kodu. [1]",
"component": ".github/workflows/ci.yml",
"type": "fullstack",
"action_type": "create",
"depends_on":,
"outputs":,
"estimated_complexity": "medium",
"acceptance_criteria": [
"Pipeline CI jest zdefiniowany w repozytorium.",
"Pipeline uruchamia się automatycznie po pushu do gałęzi main lub dev.",
"Pipeline pomyślnie wykonuje kroki: install dependencies , lint , build dla obu
aplikacji.",
"W przypadku błędu w którymkolwiek kroku, pipeline kończy się
niepowodzeniem."
]
},
{
"task_id": "BE-CORE-DB-PRISMA-SETUP",
"title": "Konfiguracja Prisma ORM i definicja schematu bazy danych",
"description": "Zainicjowanie Prisma w aplikacji backendowej. Stworzenie pliku
schema.prisma definiującego modele User i Tip zgodnie ze specyfikacją, w tym
wszystkie pola, relacje i ograniczenia (np. unikalność pól email , username ,
googleId ). [1]",
"component": "apps/backend/prisma/schema.prisma",
"type": "backend",
"action_type": "create",
"depends_on":,
"outputs":,
"estimated_complexity": "medium",
"acceptance_criteria":
},
{

```

```

"task_id": "BE-CORE-DB-MIGRATION",
"title": "Wygenerowanie i zastosowanie pierwszej migracji bazy danych",
"description": "Użycie komendy prisma migrate dev do wygenerowania pierwszej migracji SQL na podstawie schema.prisma i zastosowania jej na lokalnej bazie danych PostgreSQL działającej w Dockerze. Tworzy to fizyczną strukturę tabel w bazie. [1]",
"component": "apps/backend/prisma/migrations",
"type": "backend",
"action_type": "configure",
"depends_on":,
"outputs":,
"estimated_complexity": "low",
"acceptance_criteria":
},
{
"task_id": "BE-CORE-CIRCLE-SERVICE-SETUP",
"title": "Implementacja szkieletu CircleService do integracji z Circle API",
"description": "Stworzenie modułu CircleModule i serwisu CircleService w NestJS. Serwis ten będzie kapsułkował całą logikę komunikacji z Circle API. Na tym etapie należy zaimplementować inicjalizację SDK Circle z kluczami API (zmienne środowiskowe) i podstawową obsługę błędów. [1]",
"component": "apps/backend/src/circle/circle.service.ts",
"type": "backend",
"action_type": "create",
"depends_on":,
"outputs":,
"estimated_complexity": "medium",
"acceptance_criteria":
},
{
"task_id": "BE-CORE-QUEUE-SETUP",
"title": "Konfiguracja systemu kolejek zadań (BullMQ + Redis)",
"description": "Zintegrowanie biblioteki BullMQ z aplikacją NestJS w celu obsługi zadań w tle, takich jak asynchroniczne tworzenie portfeli Circle czy wysyłanie powiadomień. Wymaga to połączenia z serwerem Redis działającym w kontenerze Docker. [1]",
"component": "apps/backend/src/queue/queue.module.ts",
"type": "backend",

```

```

"action_type": "configure",
"depends_on":,
"outputs":,
"estimated_complexity": "medium",
"acceptance_criteria":
},
{
"task_id": "BE-AUTH-MODULE-SETUP",
"title": "Stworzenie modułu AuthModule i konfiguracja JWT",
"description": "Utworzenie AuthModule w NestJS, który będzie odpowiedzialny za całą logikę uwierzytelniania. Skonfigurowanie modułu @nestjs/jwt z sekretem JWT pobieranym ze zmiennych środowiskowych. Zdefiniowanie AuthService z metodą login(user) do generowania tokenów JWT. [1]",
"component": "apps/backend/src/auth/auth.module.ts",
"type": "backend",
"action_type": "create",
"depends_on":,
"outputs":,
"estimated_complexity": "medium",
"acceptance_criteria":
},
{
"task_id": "FE-AUTH-LOGIN-PAGE-UI",
"title": "Stworzenie strony logowania/rejestracji z opcjami",
"description": "Zaprojektowanie i zaimplementowanie komponentu strony logowania, która będzie dostępna pod adresem /login. Strona powinna zawierać przyciski 'Zaloguj przez Google', 'Zaloguj przez Twitch' (placeholder) oraz 'Zaloguj portfelem Web3'. [1]",
"component": "apps/frontend/pages/login.tsx",
"type": "frontend",
"action_type": "create",
"depends_on":,
"outputs":,
"estimated_complexity": "low",
"acceptance_criteria":
},
{
"task_id": "BE-AUTH-GOOGLE-STRATEGY",

```

```

"title": "Implementacja strategii logowania przez Google (Passport.js)",
"description": "Stworzenie GoogleStrategy w module Auth. Strategia ta, po pomyślnej autoryzacji przez Google, otrzyma profil użytkownika i przekaże go do metody validate, która z kolei wywoła AuthService.validateOAuthUser. [1]",
"component": "apps/backend/src/auth/strategies/google.strategy.ts",
"type": "backend",
"action_type": "create",
"depends_on":,
"outputs":,
"estimated_complexity": "medium",
"acceptance_criteria":
},
{
"task_id": "BE-AUTH-VALIDATE-OAUTH-USER",
"title": "Implementacja logiki validateOAuthUser w AuthService",
"description": "Stworzenie w AuthService metody validateOAuthUser, która przyjmuje dane z profilu OAuth. Metoda sprawdza, czy użytkownik z danym providerId lub email już istnieje w bazie. Jeśli nie, tworzy nowego użytkownika i emituje zdarzenie user.created do kolejki w celu utworzenia portfela. [1]",
"component": "apps/backend/src/auth/auth.service.ts",
"type": "backend",
"action_type": "refactor",
"depends_on":,
"outputs":,
"estimated_complexity": "medium",
"acceptance_criteria":
},
{
"task_id": "BE-AUTH-GOOGLE-CONTROLLER",
"title": "Stworzenie endpointów /auth/google i /auth/google/callback",
"description": "W AuthController zaimplementowanie dwóch endpointów chronionych przez AuthGuard('google'). Endpoint /auth/google zainicjuje przekierowanie do Google. Endpoint /auth/google/callback obsłuży powrót, wygeneruje JWT dla zalogowanego użytkownika (req.user) i ustawi go w ciasteczku HttpOnly, a następnie przekieruje na frontendowy dashboard. [1]",
"component": "apps/backend/src/auth/auth.controller.ts",
"type": "backend",
"action_type": "create",

```

```

"depends_on":,
"outputs":,
"estimated_complexity": "low",
"acceptance_criteria":
},
{
"task_id": "FE-AUTH-GOOGLE-BUTTON",
"title": "Implementacja przycisku 'Zaloguj przez Google' na frontendzie",
"description": "Stworzenie komponentu przycisku, który po kliknięciu
przekierowuje użytkownika na backendowy endpoint /api/auth/google.
Komponent powinien być zgodny z wytycznymi brandingowymi Google. [1]",
"component": "apps/frontend/components/auth/GoogleLoginButton.tsx",
"type": "frontend",
"action_type": "create",
"depends_on":,
"outputs":,
"estimated_complexity": "low",
"acceptance_criteria": [
"Komponent renderuje przycisk z logo i tekstem 'Zaloguj przez Google'.",
"Kliknięcie przycisku powoduje nawigację do https://api.tipjar.com/auth/google
(lub odpowiednika lokalnego).",
"Przycisk jest ostylowany zgodnie z design systemem."
]
},
{
"task_id": "FS-AUTH-GOOGLE-CONNECT",
"title": "Pełna integracja i testowanie przepływu logowania przez Google",
"description": "Połączenie komponentu frontendowego z backendem i
przetestowanie całego przepływu: kliknięcie przycisku, autoryzacja w Google,
powrót do aplikacji, ustawienie ciasteczka i przekierowanie na dashboard.
Wymaga to również implementacji na frontendzie logiki rozpoznawania stanu
zalogowania (np. poprzez sprawdzanie istnienia ciasteczka lub dedykowany
endpoint /auth/me).",
"component": "E2E Test",
"type": "integration",
"action_type": "connect",
"depends_on":,
"outputs":,

```

```

"estimated_complexity": "medium",
"acceptance_criteria": [
  "Użytkownik niezalogowany po wejściu na /dashboard jest przekierowywany na /login.",
  "Po pomyślnym zalogowaniu przez Google, użytkownik ląduje na /dashboard i jest rozpoznawany jako zalogowany.",
  "Na backendzie w bazie danych pojawia się nowy użytkownik (przy pierwszym logowaniu).",
  "W konsoli backendu widoczny jest log o wysłaniu zdarzenia user.created."
]
},
{
  "task_id": "BE-WALLET-PROVISIONING-WORKER",
  "title": "Implementacja workera kolejki do tworzenia portfeli Circle",
  "description": "Stworzenie workera BullMQ, który nasłuchuje na zdarzenia user.created. Po otrzymaniu zadania, worker wywołuje CircleService.provisionUserWallet, przekazując userId i email. Obsługuje logikę ponawiania prób w razie błędów. [1]",
  "component": "apps/backend/src/queue/wallet.worker.ts",
  "type": "backend",
  "action_type": "create",
  "depends_on":,
  "outputs":,
  "estimated_complexity": "medium",
  "acceptance_criteria":
},
{
  "task_id": "BE-WALLET-PROVISIONING-LOGIC",
  "title": "Implementacja logiki provisionUserWallet w CircleService",
  "description": "Zaimplementowanie metody provisionUserWallet w CircleService. Metoda ta wykonuje idempotentne wywołanie API Circle ( wallets.createWallet ) w celu utworzenia portfela typu SCA na sieci Polygon. Po pomyślnym utworzeniu, zapisuje circleWalletId i mainWalletAddress w rekordzie użytkownika w bazie danych. [1]",
  "component": "apps/backend/src/circle/circle.service.ts",
  "type": "backend",
  "action_type": "refactor",
  "depends_on":,

```



```

"outputs":,
"estimated_complexity": "medium",
"acceptance_criteria": ).", "Po otrzymaniu odpowiedzi z Circle, pola
circleWalletId i mainWalletAddress są poprawnie aktualizowane w bazie danych dla
danego użytkownika.", "W przypadku błędu API, jest on logowany, a worker otrzymuje
informację o niepowodzeniu." ] }, { "task_id": "FE-DASHBOARD-
SKELETON", "title": "Stworzenie szkieletu panelu twórcy (Dashboard)",
"description": "Utworzenie chronionej strony /dashboard, dostępnej tylko dla
zalogowanych użytkowników. Strona powinna zawierać podstawowy layout z nawigacją boczną
(sidebar) i miejscem na treść. Nawigacja powinna zawierać linki do: Podsumowanie, Historia
transakcji, Wpłaty, Edycja profilu. [1]", "component":
"apps/frontend/pages/dashboard/index.tsx", "type": "frontend", "action_type":
"create", "depends_on":, "outputs":, "estimated_complexity": "medium",
"acceptance_criteria": [ "Niezalogowany użytkownik próbujący wejść na
/dashboard jest przekierowywany na /login .", "Zalogowany użytkownik widzi
layout panelu z nawigacją.", "Kliknięcie linków w nawigacji prowadzi do
odpowiednich podstron (np. /dashboard/payouts )." ] }, {
"task_id": "FE-DASHBOARD-WALLET-STATUS-UI", "title": "Implementacja UI dla statusu
portfela w panelu twórcy", "description": "W panelu twórcy, w widoku podsumowania,
zaimplementowanie logiki, która sprawdza status portfela użytkownika. Jeśli portfel jest w
trakcie tworzenia ( walletStatus: 'provisioning' ), wyświetlany jest komunikat 'Twój
portfel jest tworzony...'. Po pomyślnym utworzeniu, wyświetlane jest saldo. Wymaga to
rozszerzenia endpointu /auth/me o status portfela. [1]", "component":
"apps/frontend/components/dashboard/WalletStatus.tsx", "type": "frontend",
"action_type": "create", "depends_on":, "outputs":,
"estimated_complexity": "medium", "acceptance_criteria": }, {
"task_id": "BE-PROFILE-API-PUBLIC", "title": "Stworzenie publicznego endpointu API
do pobierania danych profilu twórcy", "description": "Implementacja publicznego,
niezabezpieczonego endpointu GET /api/profiles/:username, który zwraca dane niezbędne
do wyświetlenia na stronie profilowej twórcy: displayName, avatarUrl,
bannerUrl, bio, goalAmount, goalDescription oraz mainWalletAddress.
[1]", "component": "apps/backend/src/users/users.controller.ts", "type":
"backend", "action_type": "create", "depends_on":, "outputs":,
"estimated_complexity": "low", "acceptance_criteria": [ "Endpoint jest
dostępny publicznie bez autoryzacji.", "Wyszukuje użytkownika w bazie po polu
username .", "Zwraca tylko publiczne, bezpieczne dane (nie zwraca np.
email).", "Jeśli użytkownik nie zostanie znaleziony, zwraca status 404 Not
Found." ] }, { "task_id": "FE-PROFILE-PUBLIC-PAGE", "title":
"Implementacja dynamicznej strony publicznego profilu twórcy", "description":
"Stworzenie dynamicznej strony Next.js pod adresem
/@username (np. pages/@[username].tsx ). Strona ta na serwerze (SSR/ISR) pobiera
dane z endpointu /api/profiles/:username i renderuje profil twórcy, w tym jego avatar,
bio, cel zbiórki oraz formularz do wpłacania napiwków. [1]", "component":
"apps/frontend/pages/@[username].tsx", "type": "frontend", "action_type":
"create", "depends_on":, "outputs":, "estimated_complexity": "medium",
"acceptance_criteria": [ "Wejście na adres /@[istniejący_user] poprawnie
renderuje jego profil.", "Dane takie jak nazwa, opis, banner są widoczne.",
"Jeśli ustawiony jest cel zbiórki, widoczny jest pasek postępu.", "Wejście na
nieistniejący profil zwraca stronę 404." ] }, { "task_id": "FE-

```

```

TIPPING-FORM",      "title": "Implementacja komponentu formularza napiwków",
"description": "Stworzenie reużywalnego komponentu React TippingForm , który zawiera
pole do wpisania kwoty (lub suwak), opcjonalne pole na wiadomość oraz przyciski wyboru
metody płatności ('Zapłać krypto', 'Zapłać kartą'). Komponent ten będzie używany na
stronie profilu i w widżecie. [1]",      "component":
"apps/frontend/components/tipping/TippingForm.tsx",      "type": "frontend",
"action_type": "create",      "depends_on":,      "outputs":,
"estimated_complexity": "medium",      "acceptance_criteria":  },  {
"task_id": "BE-WEBHOOK-CIRCLE-DEPOSIT",      "title": "Implementacja webhooka dla
depozytów on-chain z Circle",      "description": "Stworzenie endpointu POST
/api/webhooks/circle/deposits , który będzie odbierał powiadomienia od Circle o
nowych transakcjach przychodzących na portfele custodial twórców. Handler webhooka musi
zweryfikować sygnaturę żądania, a następnie na podstawie danych z powiadomienia (kwota,
walletId , txHash ) utworzyć nowy rekord w tabeli Tip . [1]",      "component":
"apps/backend/src/webhooks/webhooks.controller.ts",      "type": "backend",
"action_type": "create",      "depends_on":,      "outputs":,
"estimated_complexity": "high",      "acceptance_criteria":  },  {      "task_id":
"FE-TIPPING-ONCHAIN-TX",      "title": "Implementacja logiki płatności krypto on-chain
(MetaMask)",      "description": "Po kliknięciu 'Zapłać krypto' w TippingForm ,
frontend wykrywa MetaMask, prosi o połączenie, pobiera adres twórcy z danych strony, a
następnie używa ethers.js do przygotowania i wysłania transakcji transfer kontraktu USDC.
Wyświetla użytkownikowi informację o statusie transakcji (oczekująca, potwierdzona). [1]",
"component": "apps/frontend/components/tipping/TippingForm.tsx",      "type": "frontend",
"action_type": "connect",      "depends_on":,      "outputs":,
"estimated_complexity": "high",      "acceptance_criteria":  },  {      "task_id":
"BE-TIPS-INITIATE-CARD",      "title": "Implementacja endpointu do inicjowania płatności
kartą",      "description": "Stworzenie endpointu POST /api/tips/initiate-card , który
przyjmuje kwotę i creatorId . Endpoint ten komunikuje się z Circle Payments API w celu
utworzenia PaymentIntent . Zwraca do frontendu clientSecret lub inne dane potrzebne
do uruchomienia elementu płatniczego Circle. [1]",      "component":
"apps/backend/src/tips/tips.controller.ts",      "type": "backend",      "action_type":
"create",      "depends_on":,      "outputs":,      "estimated_complexity": "medium",
"acceptance_criteria":  },  {      "task_id": "BE-WEBHOOK-CIRCLE-PAYMENT",
"title": "Implementacja webhooka dla płatności kartą z Circle",      "description":
"Stworzenie endpointu POST /api/webhooks/circle/payments do odbierania powiadomień o
statusie płatności kartą. Handler weryfikuje żądanie i po otrzymaniu
zdarzenia payment.confirmed , tworzy rekord w tabeli Tip na podstawie danych z
webhooka. [1]",      "component": "apps/backend/src/webhooks/webhooks.controller.ts",
"type": "backend",      "action_type": "create",      "depends_on":,      "outputs":,
"estimated_complexity": "medium",      "acceptance_criteria":  },  {
"task_id": "FE-TIPPING-CARD-FORM",      "title": "Integracja formularza płatności kartą z
Circle Payments",      "description": "Po kliknięciu 'Zapłać kartą', frontend wywołuje
backendowy endpoint /api/tips/initiate-card , a następnie używa otrzymanych danych do
zainicjowania elementu UI Circle Payments (np. Drop-in). Obsługuje cały proces płatności,
w tym ewentualne 3D Secure, i wyświetla finalny status. [1]",      "component":
"apps/frontend/components/tipping/TippingForm.tsx",      "type": "frontend",
"action_type": "connect",      "depends_on":,      "outputs":,
"estimated_complexity": "high",      "acceptance_criteria":  },  {      "task_id":
"E2E-TIPPING-CARD-FLOW",      "title": "Test end-to-end przepływu napiwku kartą
płatniczą",      "description": "Kompleksowy test całego procesu: fan na stronie profilu
twórcy wybiera płatność kartą, przechodzi przez proces płatności w środowisku testowym

```

```

Circle, a po pomyślnej transakcji w panelu twórcy pojawia się nowy napiwek.",
"component": "E2E Test", "type": "integration", "action_type": "connect",
"depends_on":, "outputs":, "estimated_complexity": "medium",
"acceptance_criteria": }, { "task_id": "FE-DASHBOARD-HISTORY",
"title": "Implementacja widoku historii transakcji w panelu twórcy", "description":
"Stworzenie strony /dashboard/history , która pobiera z backendu listę otrzymanych
napiwków i wyświetla je w formie tabeli lub listy kart. Widok powinien zawierać informacje
o kwocie, dacie, nadawcy (jeśli nieanonimowy) i wiadomości. [1]", "component":
"apps/frontend/pages/dashboard/history.tsx", "type": "frontend",
"action_type": "create", "depends_on":, "outputs":,
"estimated_complexity": "medium", "acceptance_criteria": }, {
"task_id": "BE-TIPS-API-HISTORY", "title": "Stworzenie endpointu API do pobierania
historii napiwków", "description": "Implementacja chronionego endpointu GET
/api/tips/history , który zwraca listę napiwków dla zalogowanego użytkownika (twórcy).
Endpoint powinien wspierać paginację. [1]", "component":
"apps/backend/src/tips/tips.controller.ts", "type": "backend", "action_type":
"create", "depends_on":, "outputs":, "estimated_complexity": "low",
"acceptance_criteria": }, { "task_id": "FE-DASHBOARD-PAYOUT-FORM",
"title": "Implementacja formularza wypłaty środków w panelu twórcy", "description":
"Stworzenie strony /dashboard/payouts z formularzem, który pozwala twórcy zlecić
wypłatę środków. Formularz zawiera pole na kwotę oraz pole na adres portfela krypto. [1]",
"component": "apps/frontend/pages/dashboard/payouts.tsx", "type": "frontend",
"action_type": "create", "depends_on":, "outputs":,
"estimated_complexity": "medium", "acceptance_criteria": [ "Formularz jest
poprawnie renderowany.", "Zawiera walidację wprowadzanych danych (np. minimalna
kwota, poprawny format adresu Ethereum).", "Przycisk 'Wypłać' jest nieaktywny,
dopóki formularz nie jest poprawnie wypełniony." ] }, { "task_id":
"BE-PAYOUTS-CRYPTO-EXECUTE", "title": "Implementacja endpointu do realizacji wypłat
krypto", "description": "Stworzenie chronionego endpointu
POST /api/payouts/crypto . Endpoint weryfikuje saldo twórcy, a następnie wywołuje
CircleService` w celu zlecenia transferu on-chain z portfela custodial twórcy na
podany adres zewnętrzny. Transakcja jest opłacana przez Gas Station. [1]",
"component": "apps/backend/src/payouts/payouts.controller.ts",
"type": "backend",
"action_type": "create",
"depends_on":,
"outputs":,
"estimated_complexity": "high",
"acceptance_criteria":
},
{
"task_id": "FS-PAYOUT-CRYPTO-CONNECT",
"title": "Pełna integracja i testowanie przepływu wypłaty krypto",
"description": "Połączenie frontendowego formularza z backendowym API i
przetestowanie całego procesu wypłaty w środowisku testowym (sandbox
Circle, testnet Polygon).",

```

```

"component": "E2E Test",
"type": "integration",
"action_type": "connect",
"depends_on":,
"outputs":,
"estimated_complexity": "medium",
"acceptance_criteria":
},
{
"task_id": "TASK-GO-LIVE",
"title": "Wdrożenie MVP na środowisko produkcyjne",
"description": "Finalny krok obejmujący ostatnie przygotowania i wdrożenie aplikacji na produkcję. Wymaga to konfiguracji produkcyjnych kluczy API, domeny, certyfikatów SSL oraz uruchomienia pipeline'u CD dla środowiska produkcyjnego.",
"component": "Production Environment",
"type": "fullstack",
"action_type": "configure",
"depends_on":,
"outputs":,
"estimated_complexity": "high",
"acceptance_criteria":
}
],
"metadata": {
"entry_points":,
"exit_points":,
"diagram_suggestion": {
"type": "dependency graph",
"grouping": "per feature"
},
"groupings": {
"feature": "Authentication, Tipping, Payouts, Core Infrastructure",
"user_story": "As a Creator, I can register via Google to start receiving tips. As a Fan, I can tip a creator using my credit card. As a Creator, I can withdraw my earnings to my personal crypto wallet.",
"module": "Auth, Payments, CircleIntegration, Users"
}
}

```

```
}  
}
```